Evidencia de pruebas automatizadas con Vitest

Plataforma de evaluaciones psicolaborales AquaChile — MVP frontend

Asignatura: DSY1104 Desarrollo Full Stack II · Evaluación Parcial 2

Integrantes: Diego Uribe, Job Arjel, Sebastián Diaz

Repositorio: https://github.com/MrDeguitoo/AquaChile_FSTK

1. Resumen

Indicador	Resultado
Archivos de prueba	8 de 8 aprobados
Casos de prueba	**57 de 57 aprobados**
Herramienta	Vitest 2.1.9 con entorno jsdom
Librerías de apoyo	Testing Library (React, user-event, jest-dom)
Cobertura (proveedor v8)	Ver sección 5

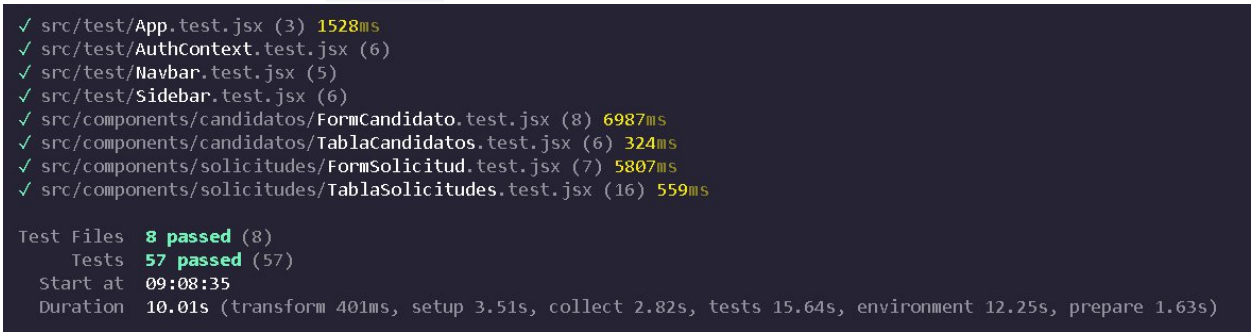
2. Configuración y ejecución

La configuración está en `vitest.config.js` (entorno `jsdom`, funciones globales habilitadas y archivo de preparación `src/test/setup.js`). Las pruebas se ubican junto a los componentes (`*.test.jsx`) o en `src/test/`.

Comandos para reproducir los resultados:

```
npm install
npm test                                # ejecuta todas las pruebas
npx vitest run --reporter=verbose       # muestra cada caso
npx vitest run --coverage               # genera el informe de cobertura
```

Captura 1 — Resultado de `npm test`:



Captura 2 — Salida con `--reporter=verbose`:

```
✓ src/test/App.test.jsx (3) 1526ms
  ✓ App - Rutas (2)
    ✓ redirige a /login si no hay sesión
    ✓ muestra formulario de login en /login
  ✓ App - Login flow (1) 1313ms
    ✓ permite login como analista y redirige a dashboard 1313ms
✓ src/test/AuthContext.test.jsx (6)
  ✓ AuthContext (6)
    ✓ provee login, logout, user y loading en el contexto
    ✓ guarda la sesión del usuario al hacer login
    ✓ limpia la sesión al hacer logout
    ✓ restaura la sesión desde localStorage al montar
    ✓ maneja localStorage corrupto limpiándolo
    ✓ inicializa user como null
✓ src/test/Navbar.test.jsx (5)
  ✓ Navbar (5)
    ✓ renderiza el título de la empresa AquaChile
    ✓ muestra el nombre y email del usuario
    ✓ llama a logout al hacer clic en el botón de cerrar sesión
    ✓ muestra botón de cerrar sesión con icono
    ✓ maneja usuario sin email
✓ src/test/Sidebar.test.jsx (6)
  ✓ Sidebar (6)
    ✓ contiene las opciones de navegación Dashboard, Candidatos y Solicitudes
    ✓ muestra badge de rol para analista
    ✓ muestra badge de rol para evaluador
    ✓ muestra badge genérico para rol desconocido
    ✓ renderiza enlaces con NavLink
    ✓ aplica clase mobile cuando mobileOpen es true
```

```
✓ FormCandidato (8) 7032ms
  ✓ renderiza todos los campos del formulario correctamente
  ✓ muestra error si faltan campos obligatorios
  ✓ muestra error si el formato de RUT es inválido 1014ms
  ✓ muestra error si el correo no es @aquachile.cl 1060ms
  ✓ rechaza un CV con extensión no permitida 1124ms
  ✓ llama a la API y onCandidatoCreated cuando todo es válido 1170ms
  ✓ muestra error si la API falla 1170ms
  ✓ muestra estado de loading mientras envía 1175ms
src/components/candidatos/TablaCandidatos.test.jsx (6)
✓ TablaCandidatos (6)
  ✓ muestra a los candidatos y el total
  ✓ invoca onSelectCandidate con el id al hacer clic en Seleccionar para Entrevista
  ✓ marca como Seleccionado solo al candidato elegido
  ✓ indica si el candidato tiene CV adjunto o no
  ✓ muestra un mensaje cuando no hay candidatos
  ✓ aplica clase de seleccionado visualmente al candidato activo
src/components/solicitudes/FormSolicitud.test.jsx (7) 5751ms
✓ FormSolicitud (7) 5751ms
  ✓ renderiza todos los campos del formulario correctamente
  ✓ muestra solo analistas (rol analista) en el selector
  ✓ muestra error si faltan campos obligatorios
  ✓ llama a la API y onSolicitudCreated cuando todo es válido 1368ms
  ✓ muestra error si la API falla 1421ms
  ✓ muestra estado de loading mientras envía 1453ms
  ✓ limpia el formulario después de crear exitosamente 1109ms
src/components/solicitudes/TablaSolicitudes.test.jsx (16) 486ms
✓ TablaSolicitudes (10) 383ms
  ✓ muestra las solicitudes y el total
  ✓ muestra los datos correctos en cada columna
  ✓ muestra select de estado para cada solicitud
  ✓ invoca onEstadoChange al cambiar el estado de una solicitud
  ✓ no invoca onEstadoChange y muestra "Guardando..." cuando está en loading
  ✓ muestra "Guardando..." mientras está en loading
  ✓ usa campos alternativos (candidatoId, ceco, fecha) si no vienen los de BD
  ✓ muestra "Sin asignar" si el candidato no existe en la lista
```

```
  ✓ muestra un mensaje cuando no hay solicitudes
  ✓ muestra todos los estados válidos en el select
✓ EstadoBadge (3)
  ✓ renderiza el badge con el estado correcto
  ✓ aplica clases CSS correctas según el estado
  ✓ usa clases por defecto para estado desconocido
✓ EstadoSelect (3)
  ✓ renderiza select con estado actual
  ✓ llama onChange al cambiar el valor
  ✓ está deshabilitado cuando disabled=true

est Files  8 passed (8)
   Tests  57 passed (57)
Start at  09:09:26
Duration  9.90s (transform 473ms, setup 3.58s, collect 2.72s, tests 15.44s, environment 11.41s, prepare 1.54s)
```

3. Casos de prueba y escenarios seleccionados

Archivo	Casos	Qué comprueba
`FormCandidato.test.jsx`	8	Validaciones del registro de candidatos: campos obligatorios, formato de RUT, correo `@aquachile.cl`, extensión del CV, y manejo de errores de la API (por ejemplo, RUT ya registrado).
`TablaCandidatos.test.jsx`	6	Renderizado del listado de candidatos con `.map()` a partir de props.
`FormSolicitud.test.jsx`	7	Formulario de solicitudes (detalle abajo).
`TablaSolicitudes.test.jsx`	16	Tabla de solicitudes, `EstadoBadge` y `EstadoSelect` (detalle abajo).
`AuthContext.test.jsx`	6	Contexto de sesión: estado inicial sin usuario, inicio y cierre de sesión, persistencia en `localStorage` y limpieza ante datos corruptos.
`App.test.jsx`	3	Flujo de acceso: login como analista y redirección al panel, rutas protegidas.
`Navbar.test.jsx`	5	Barra superior según el usuario en sesión.
`Sidebar.test.jsx`	6	Menú lateral y enlaces según el perfil.
Total	**57**	

3.1 Detalle de `FormSolicitud` (7 casos)

1. Renderiza todos los campos del formulario.
2. Muestra solo analistas (rol analista) en el selector.
3. Muestra error si faltan campos obligatorios.
4. Llama a la API y a `onSolicitudCreated` cuando todo es válido.
5. Muestra error si la API falla.
6. Muestra el estado de carga mientras envía.
7. Limpia el formulario después de crear la solicitud.

3.2 Detalle de `TablaSolicitudes`, `EstadoBadge` y `EstadoSelect` (16 casos)

`TablaSolicitudes` (10 casos):

1. Muestra las solicitudes y el total.
2. Muestra los datos correctos en cada columna.
3. Muestra un selector de estado para cada solicitud.
4. Invoca `onEstadoChange` al cambiar el estado.
5. No invoca `onEstadoChange` y muestra "Guardando..." cuando está en carga.
6. Muestra "Guardando..." mientras está en carga.

7. Usa campos alternativos (`candidatoId`, `ceco`, `fecha`) si no vienen los de la base de datos.
8. Muestra "Sin asignar" si el candidato no existe en la lista.
9. Muestra un mensaje cuando no hay solicitudes.
10. Muestra todos los estados válidos en el selector.

`EstadoBadge` (3 casos): renderiza el estado correcto, aplica las clases según el estado y usa clases por defecto ante un estado desconocido.

`EstadoSelect` (3 casos): renderiza el estado actual, llama a `onChange` al cambiar el valor y se deshabilita cuando corresponde.

3.3 Justificación de los escenarios

Se priorizaron los comportamientos que sostienen el proceso del proyecto VcM:

- **Validaciones de formularios (candidato y solicitud):** son las reglas de negocio que evitan datos erróneos en un proceso con información sensible. Se prueban tanto el caso válido como los inválidos.
- **Actualización de estado de solicitudes:** es la función de seguimiento; se verifica que el cambio se invoque, que exista retroalimentación visual ("Guardando...") y que no se dispare mientras hay una operación en curso.
- **Manejo de errores de la API:** el usuario debe ver un mensaje claro cuando algo falla, sin que la aplicación se rompa.
- **Sesión y rutas protegidas:** controlan el acceso a las vistas internas y la persistencia al recargar; incluye el caso de datos corruptos en `localStorage`.
- **Renderizado de listas con `.map()` y valores por defecto:** garantizan que las tablas muestren los datos esperados y manejen lista vacía o datos faltantes.

4. Resultados de la ejecución

```
Test Files  8 passed (8)
     Tests  57 passed (57)
```

Durante la ejecución aparecen mensajes en consola (por ejemplo, "Error al crear candidato: El RUT ya está registrado" o "Sesión corrupta, limpiando localStorage"). **No son fallos:** corresponden a casos que simulan deliberadamente un error para comprobar que la aplicación lo maneja.

5. Cobertura

Resultados de `npx vitest run --coverage` (proveedor v8) sobre el código del frontend:

Módulo	Sentencias	Ramas	Funciones	Líneas
`components/candidatos`	100%	96.1%	93.3%	100%
`components/solicitudes`	97.0%	89.9%	100%	97.0%
`FormSolicitud.jsx`	89.0%	72.2%	100%	89.0%
`Navbar.jsx`	100%	100%	100%	100%

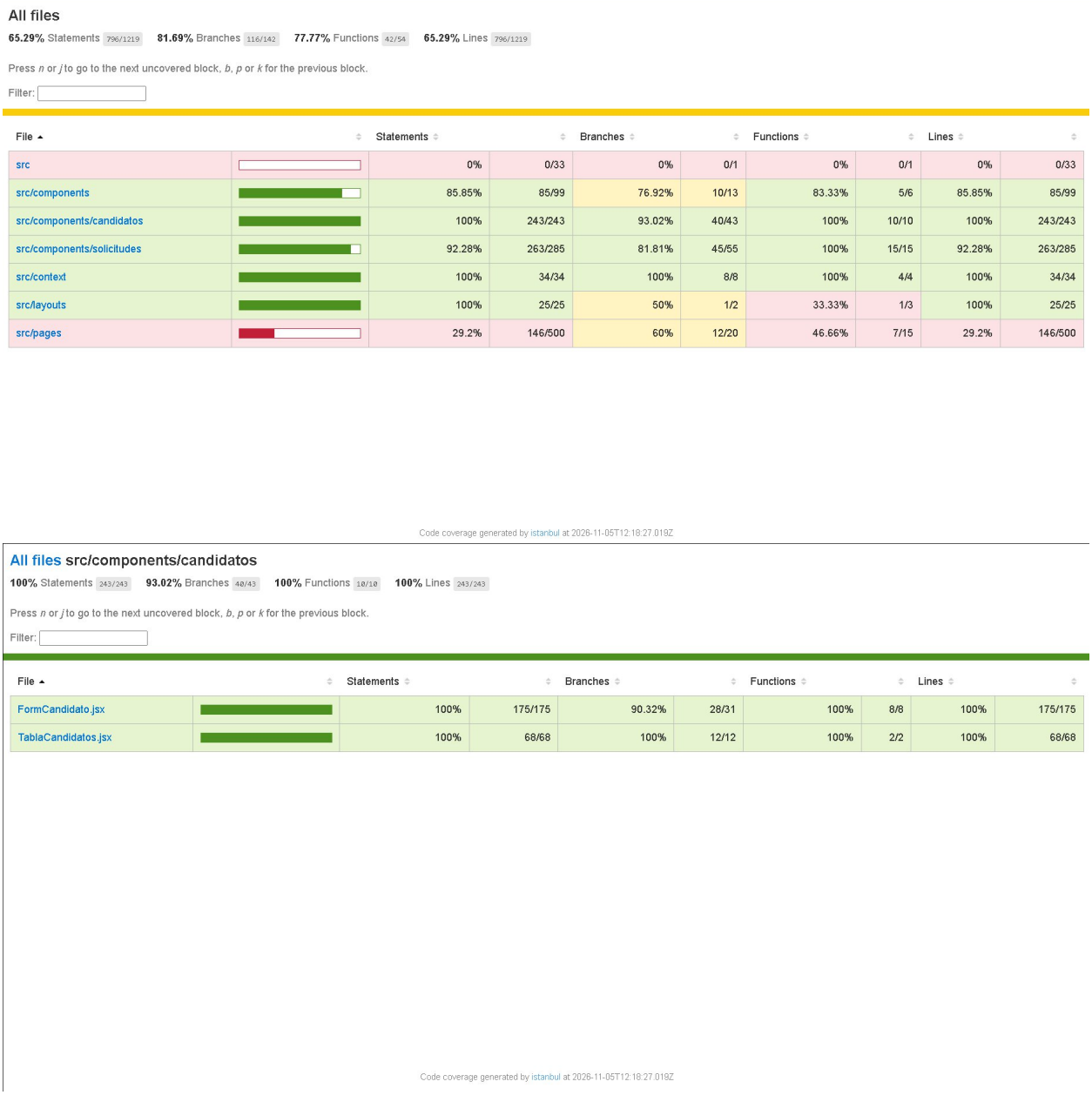
Módulo	Sentencias	Ramas	Funciones	Líneas
`Sidebar.jsx`	100%	87.5%	100%	100%
`AuthContext.jsx`	100%	100%	100%	100%
`AppLayout.jsx`	100%	50%	33.3%	100%
`login.jsx`	92.7%	61.5%	100%	92.7%
`NotFoundPage.jsx`	0%	0%	0%	0%

Nota: el total global que muestra Vitest (69.5% de líneas) incluye el archivo del servidor `server/src/index.js`, que no es parte del frontend y no se prueba en esta etapa. Por eso se reportan los módulos del frontend por separado.

Captura 3 — Tabla de cobertura en consola:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	65.29	81.69	77.77	65.29	
src	0	0	0	0	
App.jsx	0	0	0	0	1-33
src/components	85.85	76.92	83.33	85.85	
LoadingSpinner.jsx	0	100	0	0	1-8
Navbar.jsx	100	100	100	100	
ProtectedRoute.jsx	50	33.33	100	50	9-14,17-18
Sidebar.jsx	100	87.5	100	100	11
src/components/candidatos	100	93.02	100	100	
FormCandidato.jsx	100	90.32	100	100	74,91,188
TablaCandidatos.jsx	100	100	100	100	
src/components/solicitudes	92.28	81.81	100	92.28	
FormSolicitud.jsx	89	72.22	100	89	18-25,29-36,41-42,47-48,105,109
TablaSolicitudes.jsx	100	100	100	100	
src/context	100	100	100	100	
AuthContext.jsx	100	100	100	100	
src/layouts	100	50	33.33	100	
AppLayout.jsx	100	50	33.33	100	21
src/pages	29.2	60	46.66	29.2	
CandidatosPage.jsx	2.22	100	0	2.22	8-110
DashboardPage.jsx	41.93	66.66	66.66	41.93	18-33,51-52,82-145
NotFoundPage.jsx	0	0	0	0	1-23
SolicitudesPage.jsx	1.79	100	0	1.79	10-198
login.jsx	92.7	61.53	100	92.7	28-29,39,54-57

Captura 4 — Informe HTML (`coverage/index.html`):



All files

65.29% Statements 796/1219 **81.69%** Branches 116/142 **77.77%** Functions 42/54 **65.29%** Lines 796/1219

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	0%	0/33	0%	0/1	0%	0/1	0%	0/33
src/components	85.85%	85/99	76.92%	10/13	83.33%	5/6	85.85%	85/99
src/components/candidatos	100%	243/243	93.02%	40/43	100%	10/10	100%	243/243
src/components/solicitudes	92.28%	263/285	81.81%	45/55	100%	15/15	92.28%	263/285
src/context	100%	34/34	100%	8/8	100%	4/4	100%	34/34
src/layouts	100%	25/25	50%	1/2	33.33%	1/3	100%	25/25
src/pages	29.2%	146/500	60%	12/20	46.66%	7/15	29.2%	146/500

Code coverage generated by istanbul at 2026-11-05T12:18:27.019Z

All files src/components/candidatos

100% Statements 243/243 **93.02%** Branches 40/43 **100%** Functions 10/10 **100%** Lines 243/243

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
FormCandidato.jsx	100%	175/175	90.32%	28/31	100%	8/8	100%	175/175
TablaCandidatos.jsx	100%	68/68	100%	12/12	100%	2/2	100%	68/68

Code coverage generated by istanbul at 2026-11-05T12:18:27.019Z

6. Análisis

Fortalezas

- Los formularios, las tablas y la sesión, que concentran la lógica del flujo, tienen cobertura alta y prueban casos de éxito y de error.
- Las pruebas son independientes de un backend real: la API se simula, por lo que se ejecutan de forma rápida y repetible.
- La suite se ejecuta con un solo comando (`npm test`) y no requiere un navegador instalado.

Limitaciones

- Las páginas `DashboardPage`, `CandidatosPage` y `SolicitudesPage` no tienen pruebas propias; solo se ejercitan de forma indirecta en el flujo de `App.test.jsx`.
- `NotFoundPage` no está cubierta.
- Algunas ramas de `login.jsx` y `AppLayout.jsx` no se prueban.
- Aún no existe la búsqueda ni el filtrado, por lo que no hay pruebas para ellos.

Próximas pruebas

1. Pruebas de las páginas con la API simulada (carga, error y reintento).
2. Prueba de `NotFoundPage`.
3. Pruebas de búsqueda y filtros cuando se implementen.
4. Pruebas del cambio de estado de extremo a extremo (formulario, tabla y actualización).